```
# 04_train_models.py
# Trains the four experiments (A, B, C, D) with PyTorch. Needs a GPU:
# Colab > Runtime > Change runtime type > T4 GPU.
# Rough time on a T4 with all ~40k tiles: 25-40 minutes per experiment (set SUBSET in 02
# or lower EPOCHS to go faster). Finished experiments are skipped if you re-run.
```

```
# %% [1] Imports and settings
import os, json, time, random
import numpy as np
import torch
import torch.nn as nn
import torchvision
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from sklearn.metrics import fbeta_score

ROOT = os.environ.get("AMAZON_ROOT", "/content/drive/MyDrive/amazon_project")
try:
    from google.colab import drive
    drive.mount("/content/drive")
except ImportError:
    pass

CACHE, OUT = f"{ROOT}/cache", f"{ROOT}/runs"
os.makedirs(OUT, exist_ok=True)
DEV = "cuda" if torch.cuda.is_available() else "cpu"
print("Device:", DEV)
if DEV != "cuda":
    print("WARNING: no GPU. Training will be extremely slow. Switch the runtime to a GPU.")

EPOCHS, BATCH, SEED = 14, 128, 42
OVERSAMPLE, TTA = True, True
# Cached channel order is [R, G, B, NIR, NDVI] -> RGB = [0,1,2]; all = [0,1,2,3,4]
EXPERIMENTS = {
    "A_scratch_rgb":      dict(arch="scratch", chans=[0, 1, 2], lr=3e-3),
    "B_scratch_nir_ndvi": dict(arch="scratch", chans=[0, 1, 2, 3, 4], lr=3e-3),
    "C_resnet_rgb":       dict(arch="resnet", chans=[0, 1, 2], lr=1e-3),
    "D_resnet_nir_ndvi":  dict(arch="resnet", chans=[0, 1, 2, 3, 4], lr=1e-3),
}


def seed_all(s):
    random.seed(s)
    np.random.seed(s)
    torch.manual_seed(s)
    torch.cuda.manual_seed_all(s)
```

```
# %% [2] Load the cache
X = np.load(f"{CACHE}/X.npy", mmap_mode="r")
Y = np.load(f"{CACHE}/Y.npy")
sp = np.load(f"{CACHE}/splits.npz")
TR, VA, TE = sp["train"], sp["val"], sp["test"]
st = np.load(f"{CACHE}/stats.npz")
MEAN, STD = st["mean"], st["std"]
LABELS = json.load(open(f"{CACHE}/labels.json"))
NL = len(LABELS)


def load_split(idx, chans, chunk=2000):
    """Copy the chosen channels of the chosen tiles into RAM (float16)."""
    out = torch.empty((len(idx), len(chans), X.shape[2], X.shape[3]), dtype=torch.float16)
    for s in range(0, len(idx), chunk):
        part = np.asarray(X[idx[s:s + chunk]])[:, chans]
        out[s:s + chunk] = torch.from_numpy(part)
    return out
```

```
# %% [3] Models
def block(i, o):
    return nn.Sequential(
        nn.Conv2d(i, o, 3, padding=1, bias=False), nn.BatchNorm2d(o), nn.ReLU(True),
        nn.Conv2d(o, o, 3, padding=1, bias=False), nn.BatchNorm2d(o), nn.ReLU(True),
        nn.MaxPool2d(2))


class ScratchCNN(nn.Module):
    def __init__(self, c_in, n_out):
        super().__init__()
        self.features = nn.Sequential(block(c_in, 32), block(32, 64),
                                      block(64, 128), block(128, 256))
        self.head = nn.Sequential(nn.AdaptiveAvgPool2d(1), nn.Flatten(),
                                  nn.Dropout(0.3), nn.Linear(256, n_out))

    def forward(self, x):
        return self.head(self.features(x))


def build_model(arch, c_in):
    if arch == "scratch":
        return ScratchCNN(c_in, NL)
    m = torchvision.models.resnet34(weights=torchvision.models.ResNet34_Weights.IMAGENET1K_V1)
    old = m.conv1
    new = nn.Conv2d(c_in, 64, 7, 2, 3, bias=False)
    with torch.no_grad():
        new.weight[:, :3] = old.weight                  # RGB keeps the pretrained filters
        if c_in > 3:                                    # NIR / NDVI start from the RGB mean
            extra = old.weight.mean(1, keepdim=True).repeat(1, c_in - 3, 1, 1)
            new.weight[:, 3:] = extra
    m.conv1 = new
    m.fc = nn.Linear(m.fc.in_features, NL)
    return m


# %% [4] Augmentation (rotations and flips), prediction with test-time augmentation
def dihedral(x, k):
    if k >= 4:
        x = x.flip(3)
    return torch.rot90(x, k % 4, (2, 3))


def augment(x):
    ks = torch.randint(0, 8, (x.size(0),), device=x.device)
    out = torch.empty_like(x)
    for k in range(8):
        m = ks == k
        if m.any():
            out[m] = dihedral(x[m], k)
    return out


@torch.no_grad()
def predict(model, xs, mean, std, tta=False, bs=256):
    model.eval()
    probs = []
    for s in range(0, len(xs), bs):
        xb = (xs[s:s + bs].to(DEV).float() - mean) / std
        views = range(8) if tta else [0]
        p = 0
        for k in views:
            with torch.autocast("cuda", dtype=torch.float16, enabled=DEV == "cuda"):
                p = p + torch.sigmoid(model(dihedral(xb, k)).float())
        probs.append((p / len(views)).cpu().numpy())
```

```
    return np.concatenate(probs)


def macro_f2(y, p, thr=0.2):
    return fbeta_score(y, (p >= thr).astype(int), beta=2, average="macro", zero_division=0)


# %% [5] Training function for one experiment
def run(name, cfg):
    seed_all(SEED)
    chans = cfg["chans"]
    mean = torch.tensor(MEAN[chans], device=DEV).view(1, -1, 1, 1).float()
    std = torch.tensor(STD[chans], device=DEV).view(1, -1, 1, 1).float()
    xtr, xva, xte = (load_split(i, chans) for i in (TR, VA, TE))
    ytr, yva, yte = Y[TR].astype(np.float32), Y[VA], Y[TE]
    ytr_t = torch.from_numpy(ytr)

    model = build_model(cfg["arch"], len(chans)).to(DEV)
    freq = ytr.mean(0)
    pos_w = torch.tensor(np.clip(np.sqrt((1 - freq) / freq), 1, 10), dtype=torch.float32)
    crit = nn.BCEWithLogitsLoss(pos_weight=pos_w.to(DEV))
    steps = int(np.ceil(len(TR) / BATCH))
    opt = torch.optim.AdamW(model.parameters(), lr=cfg["lr"], weight_decay=1e-4)
    sched = torch.optim.lr_scheduler.OneCycleLR(opt, max_lr=cfg["lr"], epochs=EPOCHS,
                                                steps_per_epoch=steps)
    scaler = torch.amp.GradScaler("cuda", enabled=DEV == "cuda")

    # Oversampling: tiles with rarer labels are drawn more often (square-root weighting)
    w = (ytr * (freq[None, :] ** -0.5)).max(1)
    w = torch.from_numpy(w / w.sum())
    hist, best, best_state = [], -1.0, None
    for ep in range(EPOCHS):
        model.train()
        t0 = time.time()
        order = torch.multinomial(w, len(TR), replacement=True) if OVERSAMPLE \
            else torch.randperm(len(TR))
        tot = 0.0
        for s in range(0, len(order), BATCH):
            b = order[s:s + BATCH]
            xb = (xtr[b].to(DEV).float() - mean) / std
            yb = ytr_t[b].to(DEV)
            xb = augment(xb)
            opt.zero_grad(set_to_none=True)
            with torch.autocast("cuda", dtype=torch.float16, enabled=DEV == "cuda"):
                logits = model(xb)
            loss = crit(logits.float(), yb)
            scaler.scale(loss).backward()
            scaler.step(opt)
            scaler.update()
            sched.step()
            tot += loss.item() * len(b)
        pv = predict(model, xva, mean, std)
        f2 = macro_f2(yva, pv)
        hist.append(dict(epoch=ep + 1, train_loss=tot / len(TR), val_macro_f2_at_0_2=f2))
        print(f"[{name}] epoch {ep + 1}/{EPOCHS} loss {tot / len(TR):.4f} "
              f"val macroF2@0.2 {f2:.4f} ({time.time() - t0:.0f}s)")
        if f2 > best:
            best = f2
            best_state = {k: v.detach().cpu().clone() for k, v in model.state_dict().items()}

    model.load_state_dict(best_state)
    torch.save(best_state, f"{OUT}/{name}.pt")
    pv, pt = predict(model, xva, mean, std, TTA), predict(model, xte, mean, std, TTA)
    np.savez_compressed(f"{OUT}/{name}_probs.npz", val_probs=pv, val_y=yva, test_probs=pt,
                        test_y=yte, val_idx=VA, test_idx=TE)
    json.dump(hist, open(f"{OUT}/{name}_history.json", "w"))
    fig, ax = plt.subplots(1, 2, figsize=(10, 3.5))
```

```
    ax[0].plot([h["train_loss"] for h in hist])
    ax[0].set_title("Training loss")
    ax[1].plot([h["val_macro_f2_at_0_2"] for h in hist])
    ax[1].set_title("Validation macro F2 (threshold 0.2)")
    for a in ax:
        a.set_xlabel("epoch")
    plt.tight_layout()
    plt.savefig(f"{OUT}/{name}_curves.png", dpi=160)
    plt.close()
    print(f"[{name}] best val macro F2@0.2 = {best:.4f}")
    del xtr, xva, xte, model
    torch.cuda.empty_cache()


# %% [6] Run all four experiments (skips any that already finished)
for name, cfg in EXPERIMENTS.items():
    if os.path.exists(f"{OUT}/{name}_probs.npz"):
        print("Skipping", name, "(already done)")
        continue
    run(name, cfg)
print("All done. Next: 05_evaluate.py")
```